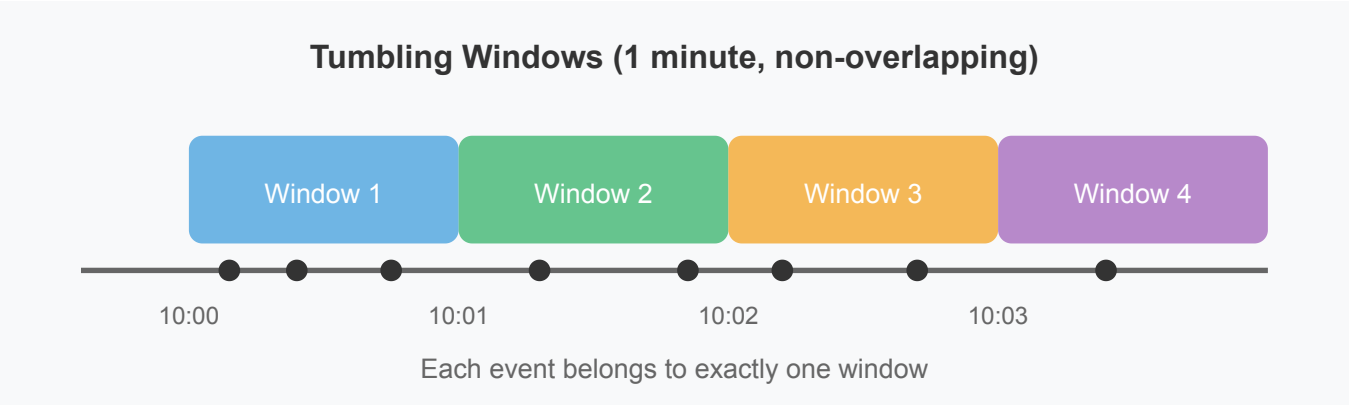
Windowing Patterns

systemdesignschool.io/fundamentals/windowing-patterns

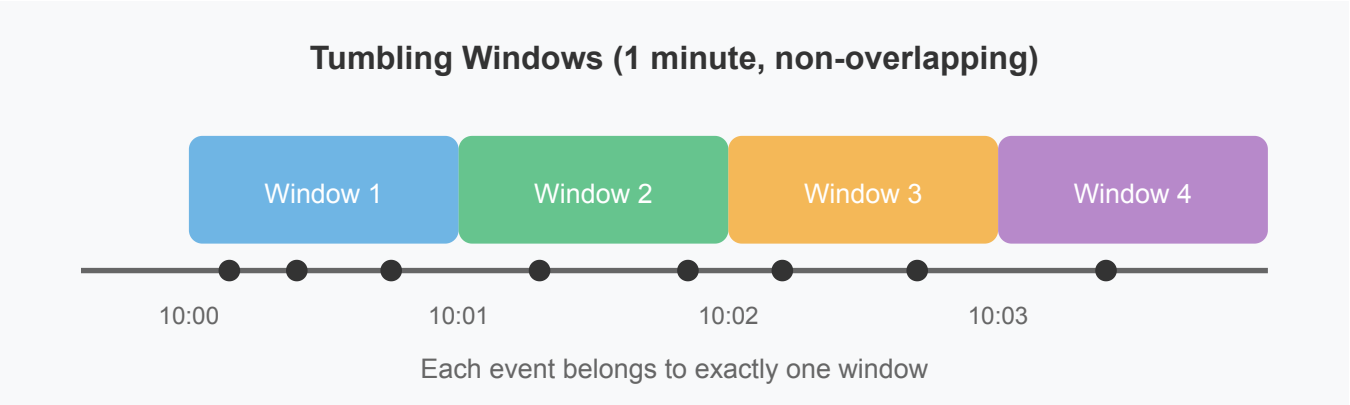


The previous article introduced stream processing as handling unbounded data. But many useful computations need aggregation: count page views per minute, sum sales per hour, detect three failed logins within 5 minutes. You can't aggregate an infinite stream—you need boundaries.

Windows solve this by dividing the continuous stream into finite chunks. Each window collects events over some interval, aggregates them, and emits a result. There are three main patterns.

Tumbling Windows: Fixed, Non-Overlapping

Tumbling windows divide time into fixed intervals that don't overlap. Each event belongs to exactly one window.



Consider counting page views per minute. A 1-minute tumbling window groups all events from 10:00:00–10:00:59, then 10:01:00–10:01:59, and so on. When the clock crosses a minute boundary, you emit the count for the completed window and start a new one.

Concrete example: An analytics dashboard shows "requests per minute" as a time-series chart. Each data point is the count from one tumbling window.

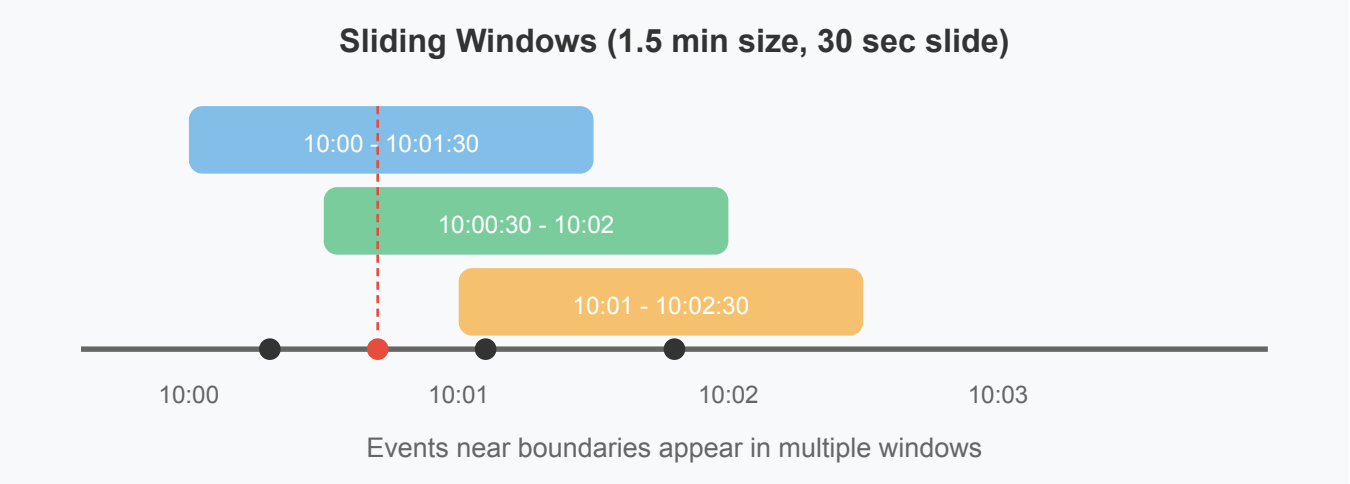
Strengths:

- Simple to implement and reason about
- Memory-efficient—you only store one window's state at a time
- Each event is counted exactly once

Limitation: Events near boundaries may split unnaturally. A user session spanning 10:59 to 11:01 contributes to two separate windows.

Sliding Windows: Overlapping for Smoothness

Sliding windows (also called hopping windows) overlap. You specify both a window size and a slide interval.



Consider a 1-minute window that slides every 30 seconds:

- Window 1: 10:00:00 – 10:00:59
- Window 2: 10:00:30 – 10:01:29
- Window 3: 10:01:00 – 10:01:59

Events between 10:00:30 and 10:00:59 appear in both Window 1 and Window 2.

Concrete example: A rate limiter checks "requests in the last 60 seconds." Using a tumbling window, a burst at 10:00:59 followed by another burst at 10:01:01 might slip through (different windows). A sliding window catches both bursts because they appear in the same overlapping window.

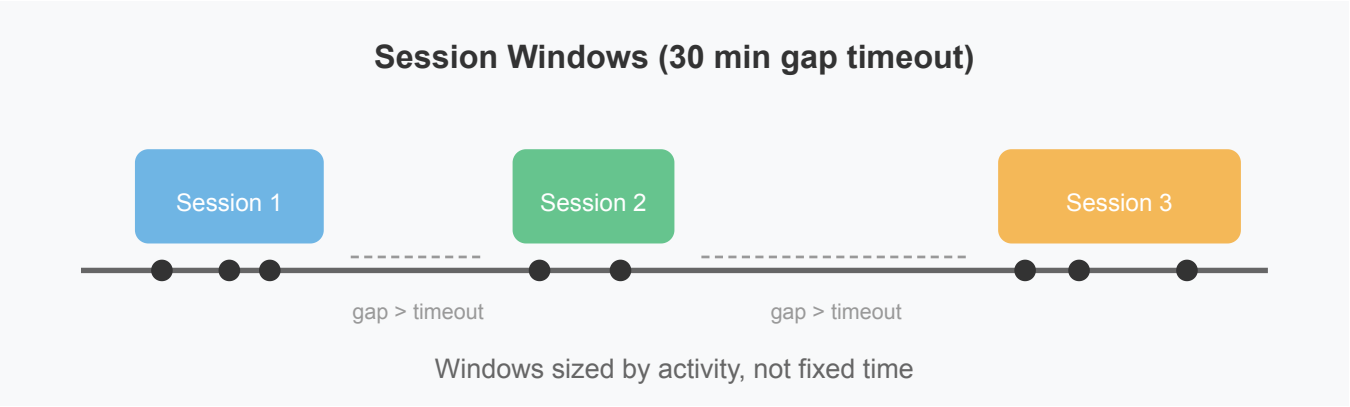
Strengths:

- Smoother metrics without boundary artifacts
- Better for detecting patterns that might straddle tumbling boundaries
- Good for moving averages

Limitation: Higher memory usage because you maintain multiple overlapping windows simultaneously. With a 1-minute window sliding every 10 seconds, you maintain 6 active windows.

Session Windows: Activity-Based

Session windows don't use fixed time intervals. They group events based on activity gaps.



Define a gap timeout (say, 30 minutes of inactivity). Events arriving within the timeout extend the current session. When no events arrive for 30 minutes, the session closes and emits.

Concrete example: Analyzing user browsing sessions on an e-commerce site. User A browses for 5 minutes, then leaves. User B browses for 2 hours. Fixed windows would chop these sessions arbitrarily. Session windows capture each user's natural activity pattern.

Strengths:

- Captures natural activity patterns
- Window size adapts to actual user behavior
- Ideal for per-user or per-entity analysis

Limitations:

- Variable output latency—you wait for the gap before emitting
- More complex state management (per-key sessions)
- Must handle very long sessions that accumulate unbounded state

Choosing a Window Type

Pattern	Use When	Trade-off
Tumbling	Periodic reports, dashboards, simple counts	Events at boundaries may split
Sliding	Rate limiting, anomaly detection, moving averages	Higher memory for overlapping windows

Pattern	Use When	Trade-off
Session	User journeys, activity analysis, per-entity patterns	Variable latency, must wait for gap

In practice, the choice often follows from the question you're answering:

- "How many requests per minute?" → Tumbling
- "Did we exceed 100 requests in any 60-second period?" → Sliding
- "How long was this user's session?" → Session

The Incomplete Window Problem

There's a catch. When should a window close and emit its result?

For tumbling windows, you might think: "when the clock shows 10:01:00, emit the 10:00 window." But what if an event from 10:00:45 arrives at 10:01:02 due to network delay? The window already closed. This event is late.

Do you:

- Drop it? (lose accuracy)
- Reopen the window and re-emit? (update previous results)
- Include it in the current window? (wrong time attribution)

That leads into **event time** (when the event actually happened) versus **processing time** (when your system received it), and **watermarks** (signals that tell the system when it's safe to close a window).